

CS351 Progress Report

Evaluating the Robustness of ML-Based Fraud Detection via
Game Theory

Daniel Lai 5529114

January 2026

Contents

1	Introduction	1
2	Background Research	1
2.1	Machine Learning for Financial Fraud Detection	1
2.2	Adversarial Machine Learning and Evasion Strategies	2
2.3	Limitations of Static Evaluation	2
2.4	Dynamic and Game-Theoretic Perspectives	3
3	Progress to Date	4
3.1	Progress Overview	4
3.2	Technical Work Completed	4
3.2.1	Transaction Simulator (Simulator v1)	4
3.2.2	Dataset Generation	5
3.2.3	Baseline Defender Pipeline	6
3.2.4	Initial Results	6
3.3	Progress Against Original Timetable	7
4	Planned Work and Next Steps	7
4.1	Immediate Next Steps	7
4.2	Plans for Term 2 and Beyond	8
4.3	Robustness Evaluation Methodology	8
4.4	Revised Timetable	9
5	Appraisals and Reflections	9
5.1	What Has Gone Well	9
5.2	Challenges and Lessons Learned	10
5.3	Focus for Term 2	10
6	Legal, Social, Ethical and Professional Issues & Considerations	10
7	Project Management	11
7.1	Development Method	11
7.2	Tools and Workflow	11
7.3	Risk Management	11

1 Introduction

Financial fraud represents a significant and growing threat within modern digital economies. As digital payment systems become increasingly embedded in everyday life, machine-learning (ML) techniques have become central to the detection of fraudulent activity. These systems analyse large volumes of transaction data in real time, seeking to identify anomalous patterns that indicate misuse or compromise. However, in practice, fraud detection models are typically trained and evaluated on static historical datasets, implicitly assuming that the underlying data distribution remains stable.

In reality, fraud is an inherently adversarial domain. Attackers continuously adapt their strategies in response to deployed detection systems, modifying transaction patterns to evade new defences. As a result, many existing ML-based fraud detectors are vulnerable to evasion and manipulation techniques that their original designs did not anticipate. Recent work in adversarial machine learning has demonstrated that models which perform well under conventional evaluation can degrade rapidly when faced with strategically crafted inputs. This highlights a critical limitation of static benchmarking approaches and motivates the need for dynamic, adversarial evaluation frameworks.

The central aim of this project is to evaluate the robustness of machine-learning based fraud detection systems within such an adversarial setting. Rather than relying on fixed benchmark datasets, the project constructs a synthetic transaction environment in which realistic user behaviour can be simulated and controlled. Within this environment, an ML-based defender is trained to detect fraudulent transactions, while an attacker component generates evasive transaction patterns designed to bypass the model. By observing how detection performance changes under increasingly sophisticated attacks, the project seeks to characterise the vulnerabilities of common fraud detection approaches and assess their resilience.

2 Background Research

This project sits at the intersection of financial fraud detection, machine learning, and adversarial security. Existing work in these areas highlights both the effectiveness of data-driven detection systems and their limitations when deployed in adversarial environments. This section will briefly outline the key concepts that motivate the project and situates it within the wider research landscape.

2.1 Machine Learning for Financial Fraud Detection

Financial fraud represents a persistent and evolving threat in digital payment ecosystems. Machine learning has become the dominant paradigm for fraud detection, enabling institutions to process large volumes of transaction data and identify anomalous behaviour in real time.[1] Unlike rule-based systems, which are manually crafted and brittle, machine learning models learn behavioural patterns directly from historical data and can adapt as spending habits and fraud tactics evolve.[1]

A typical fraud detection pipeline transforms raw transactions into behavioural features,

such as velocity measures, temporal patterns, merchant novelty, and user-relative deviations. These features are then consumed by classifiers including logistic regression, random forests, and gradient boosting methods.[2][1] Because fraudulent transactions form only a small fraction of the overall population, evaluation relies on metrics suitable for imbalanced classification, such as precision, recall, F1-score, and precision-recall AUC.[1]

While these systems enable real-time decision-making and continual adaptation, they also introduce new vulnerabilities. Models trained on historical data expose consistent decision boundaries that can be probed and exploited by adversaries.[2] An attacker who observes system behaviour can deliberately craft transactions that evade detection, a risk that is not captured by static train-test evaluation protocols.[2]

2.2 Adversarial Machine Learning and Evasion Strategies

Adversarial machine learning studies the robustness of models when faced with strategically crafted, worst-case inputs. In fraud detection, attackers employ two primary classes of attack: *evasion* attacks, which modify individual transactions at inference time to bypass deployed classifiers, and *poisoning* attacks, which contaminate training data to corrupt model learning.[2]

Evasion attacks in fraud detection operate under realistic domain constraints. Unlike adversarial perturbations in image classification, transaction modifications must remain semantically plausible: amounts must be non-negative, spending behaviour must appear legitimate, and merchant choices must respect business logic.[2] Despite these constraints, evasion can be highly effective. Carminati et al. [3] demonstrated evasion rates between 60% and 100% against banking fraud detectors, while more recent work reports attack success rates exceeding 87% against state-of-the-art graph-based fraud models under gradient-based evasion strategies.[7]

Poisoning attacks inject malicious data into the training process to degrade model performance. Even small contamination rates can significantly reduce accuracy in fraud detection tasks,[6] and in systems that automate labelling, poisoning can create feedback loops in which corrupted models misclassify future attacks, causing the attack to propagate over time.[2]

Together, these attack classes expose a fundamental limitation of conventional evaluation protocols. Models trained and tested on static historical datasets do not reflect the adversarial environment in which fraud detection systems operate.[9][2] In practice, fraud detection is an arms race between attackers and defenders, yet standard benchmarking assumes that test distributions are fixed and non-adversarial.

2.3 Limitations of Static Evaluation

Static evaluation, in which models are trained on historical data and tested on a held-out dataset, is standard practice in machine learning. While this approach is efficient and reproducible, it implicitly assumes that the test distribution is representative of future deployment conditions and that no strategic adversary is actively shaping the data.[8] [9]

In fraud detection, this assumption rarely holds. Fraudsters observe system behaviour and modify their tactics to evade detection, giving rise to *concept drift*. [5][4] This drift may occur abruptly in response to new defences, gradually over time, or incrementally through small behavioural changes that accumulate. [5] As a result, models trained on historical data often degrade in performance after deployment. [11] Static benchmarks provide no mechanism to measure this degradation or to evaluate defences under adaptive pressure. [9]

The limitations of static evaluation are illustrated by Meta’s Dynabench project, which replaces fixed test sets with dynamic adversarial data collection. [8] Models that perform well on static benchmarks were shown to fail rapidly when faced with adaptive human adversaries. [8] Although developed for natural language processing, this insight generalises to fraud detection: a detector that appears robust on a fixed dataset may perform substantially worse against an adaptive attacker.

2.4 Dynamic and Game-Theoretic Perspectives

Addressing the limitations of static evaluation requires moving beyond fixed benchmarks to dynamic frameworks in which attackers and defenders interact strategically over time. Game theory provides a formal foundation for modelling these interactions, treating fraud detection as a sequential decision process between rational agents with competing objectives. [12][10]

In this setting, the attacker seeks to maximise evasion success while remaining within domain constraints, and the defender aims to maintain detection accuracy while controlling false positives. The interaction can be modelled as an iterative process: the defender trains a model, the attacker observes its behaviour and crafts evasive transactions, performance degradation is measured, and—under more advanced formulations—the defender retrains on detected attacks, forming an arms race.

Empirical work on game-theoretic defences demonstrates that strategic assumptions significantly affect outcomes. For example, Stackelberg formulations, in which the defender commits to a strategy and the attacker responds optimally, have been shown to achieve substantially lower misclassification rates than Nash-style equilibria. This highlights the importance of proactive defence design and anticipation of adversarial response.

Game-theoretic models also expose the role of information asymmetry and uncertainty. Attackers may operate under white-box, grey-box, or black-box assumptions, with varying query budgets and behavioural constraints. [13] Defenders, in turn, must remain robust across multiple threat models and adapt as attacker capabilities evolve.

By framing fraud detection as a dynamic strategic interaction, these approaches move beyond static evaluation and motivate experimental frameworks that iteratively pit defenders against attackers, measure performance degradation, and analyse resilience under adaptive pressure. [10] This perspective directly informs the design and objectives of this project.

3 Progress to Date

3.1 Progress Overview

The first phase of the project, as outlined in the Project Specification, focuses on constructing the foundational components required for subsequent adversarial analysis. This includes the design and implementation of a synthetic transaction simulator and the development of a baseline machine-learning based fraud detection pipeline.

The current transaction simulator is capable of generating large-scale, multi-user transaction streams exhibiting realistic temporal structure, behavioural diversity, and longitudinal drift. In parallel, a complete data processing and modelling pipeline has been implemented, enabling time-consistent training and evaluation of fraud detection models on the generated data.

These components together form an end-to-end experimental framework: synthetic transaction data is produced by the simulator, processed into behavioural features, and consumed by a machine learning defender that outputs fraud predictions. This framework now provides a stable platform for both baseline benchmarking and future adversarial experimentation.

3.2 Technical Work Completed

The primary technical contribution of the first phase of the project is the construction of an end-to-end experimental framework for studying fraud detection under adversarial conditions. This framework consists of three tightly coupled components :

- a synthetic transaction simulator
- a data processing and feature engineering pipeline
- a machine-learning based defender



Figure 1: End-to-end experimental framework from synthetic transaction generation to fraud prediction.

Together, these components enable the controlled generation of realistic transaction streams, their transformation into behavioural features, and the training and evaluation of fraud detection models under time-consistent conditions.

3.2.1 Transaction Simulator (Simulator v1)

A custom transaction simulator was implemented in Python to generate fully synthetic, yet behaviourally plausible, credit card transaction streams. The simulator provides

a controlled environment in which user behaviour, transaction patterns, and temporal structure can be systematically defined and manipulated, enabling realistic experimentation under adversarial conditions.

Users are represented as *personas*, each associated with a behavioural profile governing spending categories, temporal activity patterns, and transaction magnitudes. Each persona defines hourly probability distributions for weekdays and weekends, producing transactions that exhibit structured daily rhythms (e.g., morning coffee, midday meals, evening shopping) rather than uniformly distributed activity.

To reflect habitual consumer behaviour, each category maps to a small set of merchants, with a *merchant stickiness* mechanism causing personas to repeatedly transact with preferred merchants while occasionally switching. Routine behaviour is further modelled through recurring payments such as subscriptions, phone bills, and utilities, which occur on fixed days with small temporal jitter. Correlated spending is captured via a simple trip-based mechanism in which trigger events (e.g., grocery shopping) generate additional transactions within short time windows, producing realistic bursts of activity.

Finally, longitudinal behaviour drift is introduced through small weekly perturbations to spending parameters, ensuring that user behaviour evolves over time rather than remaining static.

This implementation represents the first working version of the simulator. While already capable of generating structured and temporally realistic data, it is intentionally lightweight and extensible. Additional personas, categories, and behavioural mechanisms can be introduced as the project progresses, allowing the data generation process to be refined in response to experimental findings.

3.2.2 Dataset Generation

Using the transaction simulator, a synthetic dataset was generated to support the training and evaluation of baseline fraud detection models. In its current configuration, the simulator produces transactions for 400 users over a 90-day period, yielding a multi-user, time-ordered transaction stream that captures both short-term behaviour and longer-term trends. This configuration can be readily scaled if required.

Each transaction follows a consistent schema comprising a unique identifier, timestamp, user and card identifiers, transaction amount, merchant identifier, merchant category, merchant country, channel, and a binary fraud label. This structure mirrors that of real-world card transaction datasets and enables processing using standard fraud detection pipelines.

A key design requirement is that all downstream modelling respects the temporal nature of the data. Transactions are emitted in strict chronological order, allowing the dataset to be split along a time boundary rather than at random. This prevents information leakage from the future into the past and more accurately reflects deployment conditions, where models must make predictions on unseen transactions.

The resulting dataset exhibits natural class imbalance, with fraudulent transactions forming a small minority of the overall population. To ensure experimental reproducibility, all stochastic components of the simulation are controlled via fixed random seeds, allowing identical datasets to be regenerated across runs and ensuring that performance differences arise from methodological changes rather than data variation.

3.2.3 Baseline Defender Pipeline

On top of the synthetic dataset, an end-to-end fraud detection pipeline was implemented using `scikit-learn`. The pipeline is designed to reflect realistic deployment conditions, in which models are trained on historical data and evaluated on future, unseen transactions.

Rather than performing a random split, the dataset is partitioned using a strict temporal boundary. Transactions before a chosen cut-off date form the training set, while subsequent transactions are reserved for testing. This prevents leakage of future information and ensures that evaluation reflects genuine out-of-sample performance.

A feature engineering stage transforms raw transaction records into behavioural representations suitable for machine learning. Features are computed incrementally using only information available prior to each transaction. These include temporal features (e.g., hour of day and day of week), velocity-based measures (e.g., transaction counts within recent windows), novelty indicators (e.g., first-time merchant or category), and user-relative deviations from historical spending patterns. Together, these capture both short-term anomalies and longer-term context.

The modelling pipeline integrates preprocessing, feature transformation, and classification into a single reproducible workflow. An initial logistic regression classifier was implemented as a baseline defender to validate the end-to-end system and establish a reference level of performance.

The modular design allows alternative classifiers to be introduced with minimal changes. More expressive models such as Random Forests and Gradient Boosting classifiers can therefore be incorporated in subsequent stages, enabling systematic comparison between defenders under both benign and adversarial conditions.

3.2.4 Initial Results

To validate the end-to-end framework, an initial set of experiments was conducted using the baseline logistic regression defender. The model was trained on the temporally ordered training set and evaluated on future transactions held out for testing. Performance was measured using metrics appropriate for imbalanced classification, including precision, recall, and the F1-score.

The baseline model achieved meaningful predictive performance, successfully identifying a proportion of fraudulent transactions while maintaining a low false positive rate. These results confirm that the synthetic dataset contains learnable structure and that the feature engineering pipeline captures behavioural patterns relevant to fraud detection. At the same time, the performance of the linear model highlights clear limitations,

particularly in its ability to model complex, non-linear interactions between features.

These findings motivate two immediate directions for subsequent work. First, more expressive classifiers will be introduced to establish stronger baseline defenders. Second, the observed performance on clean data provides a reference point against which the impact of adversarial manipulation can be measured. This establishes a concrete experimental foundation for the adversarial evaluation phase of the project.

3.3 Progress Against Original Timetable

The objectives set out in the original timetable in the Project Specification have all been met. A fully functional transaction simulator has been implemented, and an end-to-end fraud detection pipeline is now operational.

In practice, development progressed more iteratively than originally anticipated. Considerable time was spent refining the simulator to achieve a level of behavioural realism sufficient for meaningful experimentation. Early versions produced transaction streams that were overly simplistic, prompting the introduction of additional mechanisms such as persona-specific temporal patterns, merchant stickiness, recurring payments, event clustering, and longitudinal drift. While this refinement extended the simulator development phase beyond its initial estimate, it resulted in a more realistic and flexible data generation framework.

As a consequence, the implementation of the baseline defender began later than originally planned. However, this delay was mitigated by prioritising the construction of a minimal, end-to-end pipeline, allowing the simulator output to be rapidly integrated into a working feature engineering and modelling workflow. In addition, the buffer period allocated during the winter break was used to consolidate delayed work and complete the remaining components of Phase 1. This ensured that the core objectives of the first project phase were achieved within the intended timeframe.

Overall, while the internal ordering of tasks shifted, the main milestones of Phase 1 have been completed. The project is now positioned to proceed with Phase 2, with a stable experimental platform in place and will be further refined and extended to support adversarial evaluation.

4 Planned Work and Next Steps

4.1 Immediate Next Steps

With the core simulation and baseline detection pipeline now in place, the immediate focus of the project is to strengthen and stabilise the defender component. While an initial logistic regression model has been implemented to validate the end-to-end pipeline, this model serves primarily as a proof of concept rather than a strong baseline.

The next stage will therefore involve training and evaluating a small suite of more expressive machine learning models, such as Random Forests and Gradient Boosting classifier, using the existing feature engineering framework. These models are commonly used

in practical fraud detection systems and are better suited to capturing non-linear behavioural patterns. The goal of this stage is to establish a set of competitive baseline defenders whose performance on clean synthetic data is well understood and reproducible.

In parallel, the feature engineering pipeline will be refined and documented, ensuring that all behavioural features are computed in a time-consistent manner without data leakage. This includes finalising rolling-window statistics, novelty indicators, and user-relative deviation measures. At the conclusion of this phase, the project will have a robust, well-characterised defender suite that can serve as the foundation for subsequent adversarial experimentation

4.2 Plans for Term 2 and Beyond

Following the completion of a simple working baseline defender suite, the project will transition into its second phase, which focuses on adversarial intersection and robustness analysis. The central objective of this phase is to model an adaptive attacker capable of generating evasive transaction patterns designed to bypass trained fraud detection models.

The attacker will operate within the synthetic environment and manipulate transaction attributes in ways that preserve semantic plausibility while degrading model performance. Initial strategies will include controlled perturbations of transaction amounts, timing, and merchant choice, as well as feature-aware evasion techniques that explore the defender’s learned decision boundaries. These attacks are designed to emulate realistic fraud behaviour rather than arbitrary noise, ensuring that evasion remains consistent with the underlying transaction domain.

Building on this, the project will explore iterative attacker-defender interaction. In its simplest form, this consists of training a defender on clean data, applying adversarial manipulation at inference time, and measuring performance degradation. If time permits, this will be extended to include a retraining loop in which the defender is updated using detected attacks, simulating an “arms race” between attacker and defender.

This phase will transform the project from a static evaluation of fraud detection models into a dynamic experimental framework for studying robustness under strategic pressure. The outcome will be a set of empirical insights into how different model classes respond to adversarial behaviour and which design choices contribute most strongly to resilience.

4.3 Robustness Evaluation Methodology

Robustness in this project is defined as a defender’s ability to maintain reliable fraud detection performance in the presence of adversarially manipulated transactions. To quantify this, all models will be evaluated under two conditions: on clean, unmodified transaction data and on data subjected to adversarial evasion strategies.

For each defender, a baseline performance profile will first be established using the clean test set. Metrics appropriate for highly imbalanced classification will be reported, including precision, recall, F1-score, and precision-recall area under the curve (PR-AUC).

These results form a reference point against which adversarial degradation can be measured.

Adversarial evaluation will then be conducted by applying attacker strategies at inference time. The attacker modifies incoming transactions before they are passed to the defender, simulating evasive fraud behaviour. The same test set will be reused under controlled perturbations, ensuring that performance differences are attributable solely to adversarial manipulation rather than data variation.

Robustness will be assessed by measuring relative drops in detection performance under attack, for example the reduction in recall at a fixed false-positive rate. This enables direct comparison between models in terms of both absolute accuracy and resilience to evasion. Where multiple attack strategies are available, defenders will be evaluated against each in turn, allowing vulnerability profiles to be constructed.

This protocol ensures that all defenders are compared under identical conditions and that adversarial impact is measured in a reproducible and interpretable manner. The resulting experiments will provide a systematic basis for analysing how different model classes respond to strategic manipulation and which design choices contribute most strongly to robustness.

4.4 Revised Timetable

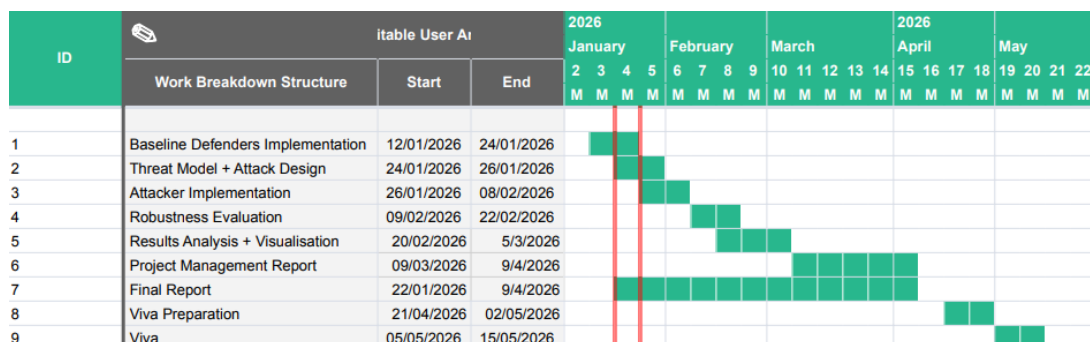


Figure 2: Term 2 Timetable

5 Appraisals and Reflections

5.1 What Has Gone Well

A key success of the project to date has been the early establishment of an end-to-end working framework. Rather than developing the simulator, data pipeline, and model in isolation, the project was structured to ensure that synthetic data could flow through the entire pipeline from generation to prediction. This approach exposed integration issues early and ensured that all components evolved in a coherent manner.

The iterative development of the transaction simulator has also proven effective. Initial versions were intentionally simple, enabling quick testing and validation. As shortcomings in behavioural realism became apparent, additional mechanisms such as temporal

profiles, merchant stickiness, recurring payments, and behavioural drift were introduced incrementally. This process produced a simulator that is both functional and extensible, while avoiding premature over-engineering.

5.2 Challenges and Lessons Learned

The most significant challenge encountered during the first phase was achieving a level of behavioural realism sufficient for meaningful experimentation. Early simulator outputs were technically correct but lacked the structure and variability required to resemble real spending behaviour. This highlighted that “realism” in synthetic data is not achieved through a single mechanism, but through the careful composition of multiple interacting effects.

Another important lesson was the importance of experimental discipline in machine learning pipelines. Issues such as temporal leakage, improper dataset splitting, and feature availability only become apparent when the system is exercised end-to-end. Addressing these problems early reinforced the necessity of time-consistent evaluation and reproducible data generation, both of which are now core design principles of the project.

5.3 Focus for Term 2

These experiences have directly shaped the approach for the remainder of the project. Future development will continue to prioritise early integration and empirical validation over isolated component design. Adversarial mechanisms will be introduced incrementally and evaluated against established baselines to ensure that their impact is measurable and interpretable.

The project has also demonstrated the value of maintaining flexibility in both design and planning. Buffer time and iterative milestones have proven essential in accommodating unforeseen complexity, particularly in areas such as behavioural modelling. This approach will be carried forward into Phase 2, ensuring that adversarial experimentation and robustness analysis remain grounded in a stable and reliable experimental framework.

6 Legal, Social, Ethical and Professional Issues & Considerations

This project evaluates machine-learning based financial fraud detection systems under adversarial conditions. All experiments are conducted within a fully synthetic environment using data generated by a custom transaction simulator. No real financial records, personal data, or human participants are involved at any stage of the project.

While the project explores adversarial techniques, these are implemented solely within a controlled academic environment and are explicitly framed as defensive research. The attacker components are designed to generate evasive transaction patterns within the simulator and are not capable of interacting with, or causing harm to, any real system. The purpose of this work is to identify vulnerabilities in fraud detection models in order to improve their robustness, not to facilitate misuse.

Findings will be presented in a way that highlights defensive implications and mitigation strategies. This ensures that the work contributes positively to the understanding of machine learning security without creating tools or guidance that could be misapplied in real-world fraud.

7 Project Management

7.1 Development Method

The project has been using an iterative and incremental approach to develop. Rather than attempting to design and implement the full system upfront, each major component was first implemented in a minimal, functional form and then progressively refined. This approach was particularly significant for the transaction simulator, where early versions were intentionally simple and used to validate assumptions before additional behavioural mechanisms were introduced.

Each iteration followed a cycle of design, implementation, testing, and refinement. This enabled immediate feedback on whether a component behaved as expected and whether it integrated cleanly with the rest of the system.

7.2 Tools and Workflow

All implementation has been carried out in Python up till now, with development split between reusable modules (e.g., the simulator and feature engineering code) and Jupyter notebooks for exploratory analysis and experimentation. This separation allows the core system to remain stable and reproducible while supporting rapid prototyping during model development.

Version control is managed using `git`, with all source code stored in a private Github repository. This enables systematic tracking of changes, rollback in the event of errors, and clear separation between experimental and stable code. Key experimental results are saved in a reproducible form, and fixed random seeds are used throughout the simulator and modelling pipeline to ensure that datasets and results can be regenerated.

Documentation of findings is also maintained alongside the codebase, and major design decisions are recorded to support later write-up in the Final Report. This workflow ensures that the project remains organised, auditable, and reproducible.

7.3 Risk Management

Several risks identified in the Project Specification have been identified in practice, most notably the challenge of achieving sufficient realism in synthetic transaction data. This risk was mitigated through more read-ups followed by iterative refinement of the simulator and by allocating buffer time during the winter break to consolidate delayed work.

Another risk, such as integration complexity and model underperformance, are addressed

by maintaining an end-to-end working system and by validating each component in context. By ensuring that the full pipeline remains operational throughout development, failures are detected early and can be resolved before they propagate.

This project management strategy emphasises early integration, incremental complexity, and continuous validation. As the project enters its adversarial phase, these practices will remain central to managing the increased experimental and analytical complexity.

References

- [1] Luis H. Aros, L. Ximena, F. Gutierrez-Portela, J. Johver, and M. Samuel. Financial fraud detection through the application of machine learning techniques: A literature review. *Humanities and Social Sciences Communications*, 11(1), 2024.
- [2] arXiv. Adversarial learning in real-world fraud detection: Challenges and perspectives. <https://arxiv.org/html/2307.01390>, 2023. Accessed: 2 November 2025.
- [3] Marco Carminati, Luca Santini, Mario Polino, and Stefano Zanero. Evasion attacks against banking fraud detection systems. In *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. USENIX Association, 2020.
- [4] Evidently AI. What is concept drift in machine learning, and how to detect and address it. <https://www.evidentlyai.com/ml-in-production/concept-drift>, 2024. Accessed: 15 January 2026.
- [5] Facctum. Frequently asked questions about concept drift. <https://www.facctum.com/terms/concept-drift>, 2025. Accessed: 13 January 2026.
- [6] Halima Kure, Prasenjit Sarkar, Alexander Nwajana, and Abubakar Ndanusa. Detecting and preventing data poisoning attacks on ai models. *arXiv preprint*, 2025.
- [7] Curthbert Jeremiah Malingu, Damilola Hannah Titilayo, Fejiro Eni, Collin Arnold Kabwama, and Vincent Anyah. Adversarial machine learning in finance: Developing resilient ai models to counter fraudster evasion attacks on us bank security systems. *International Journal of Computer Applications Technology and Research*, 2025.
- [8] Meta AI. Introducing dynabench: Rethinking the way we benchmark ai. <https://ai.meta.com/blog/dynabench-rethinking-ai-benchmarking/>, 2020. Accessed: 19 January 2026.
- [9] Amir Partovi. Dynamic benchmarking: Evaluate ai agents through environments, not datasets. <https://veris.ai/blog/dynamic-benchmarking>, 2025. Accessed: 18 January 2026.
- [10] Bhagawati Chunilal Patil. Game theory and adversarial machine learning: Analyzing strategic interactions in cybersecurity. *Communications on Applied Nonlinear Analysis*, 31(3s):470–486, 2024.
- [11] Alexander Patino. Mitigating model drift in machine learning. <https://aerospike.com/blog/model-drift-machine-learning/>, 2025. Aerospike blog.
- [12] SEI Blog. Using game theory to advance the quest for autonomous cyber threat hunting. <https://www.sei.cmu.edu/blog/Using-Game-Theory-to-Advance-Cyber-Threat-Hunting/>, 2023. Accessed: 18 January 2026.
- [13] System Overflow. What is adversarial robustness in fraud detection systems?, 2025. Accessed 20 December 2026.

Appendix A : Project Specification

Project Specification

1 Problem Statement

As technology becomes increasingly integrated into daily life, the benefits of improved convenience and service delivery are evident. However, this rapid adoption sometimes leads to overlooked security fundamentals, especially within older or legacy systems [2]. Nowhere is this more critical than in the financial sector, where the stakes are exceptionally high and sensitive information is constantly at risk [4].

The landscape of financial fraud is constantly evolving, with skilled attackers constantly developing new methods to bypass and compromise security mechanisms. Many current detection systems, built on traditional machine learning techniques, are vulnerable to advanced adversarial attacks - including poisoning (manipulating training data) and evasion (crafting inputs to deceive models) [1, 5]. As a result, these systems face threats that their original designs did not anticipate.

Evaluating fraud detection models solely on static datasets is therefore insufficient. Recent work has shown that adversarial training and dynamic evaluation frameworks can significantly enhance model robustness against worst-case perturbations, underscoring the need for adaptive defence strategies [3]. Consequently, this project will model the strategic, iterative interactions between adversaries and defenders using a game-theoretic framework, with the goal of developing more resilient and practically deployable fraud detection systems.

2 Objectives

This project aims to evaluate the robustness of machine-learning based fraud detection systems within a dynamic, adversarial context. By creating a simulated environment, it will model the strategic, iterative interaction between an adaptive attacker and an ML-based defender using a game-theoretic framework.

To achieve this, this project is broken down into a set of core deliverables and potential extensions:

2.1 Core Objectives

Phase 1: Foundational Components

1. **Define a Specific Use Case:** Research and select a specific domain of financial fraud (e.g., credit card transaction fraud, insurance claim fraud) to serve as the context for the simulation. This will define the features and patterns to be modelled.
2. **Develop a Simulation Environment:** Build a Python simulator to generate a realistic time-series dataset of legitimate financial transactions based on defined user personas.
3. **Implement a Baseline ML Defender:** Train and benchmark machine learning classifiers on the simulated data to establish an effective baseline detection model.

Phase 2: Adversarial Analysis

4. **Design an Adaptive Adversary:** Create an attacker agent capable of applying subtle, evasive attack strategies designed to bypass the defender model.
5. **Evaluate and Visualise System Robustness:** Measure the defender model's performance degradation under adversarial attacks to identify and analyse its key vulnerabilities.

2.2 Possible Extensions

1. **Apply Game-Theoretic Analysis:** Model the attacker-defender interaction as a strategic game, analysing simulation results to identify potential equilibrium strategies.
2. **Model a Defender Retraining Loop:** Implement a feedback mechanism allowing the defender to retrain on detected attacks, simulating a dynamic security "arms race".

3 Methods & Methodology

This project will use an iterative and incremental development model, along with some agile properties. This approach will combine a structured, phased development process for building the system components with a formal research framework for the experimental analysis.

3.1 Development

- The system will be built in stages. First, a simple simulator with one user persona will be developed. This will be followed by the baseline defender, and so on. This ensures that there is always a working, testable version of the system.
- Each component will be refined through a cycle of design to implementation to testing and lastly refining. Initial results may show that the simulated transactions are too simplistic. The simulator will then be refined in a new iteration to add more complexity. This iterative process would be useful as the characteristics may evolve based on initial findings

3.2 Research

- A literature review will be conducted to identify realistic user spending patterns and common evasive attack strategies. This will ensure the simulation is as realistic to real-world scenarios.
- The simulator provides a controlled environment to test the hypothesis that ML models are vulnerable to specific evasive attacks. The independent variable will be the type and intensity of the attack, and the dependent variable will be the performance of the defender model
- Performance metrics (accuracy, precision, recall, F1-score, and so on) will be compared before and after attacks. Statistical significance will be assessed where applicable. The findings from the models' performance will be statistically analysed to

draw conclusions about the model's vulnerabilities and the effectiveness of different attack strategies.

3.3 Project Management

- Git will be used for version control to manage source codes with Github to track changes and prevent data loss.
- Regular meetings and check-ins with the Project Supervisor will serve as checkpoints to discuss progress, advise and validate the project's direction.
- A logging document to track accomplishments, progress and any issues that arose during the implementation of the project's objectives.

4 Timetable

The table below shows this project’s planned rough timeline, including all of the hard deadlines for the submissions. The durations in the timetable for the whole project are thought to be overestimated to allow slack for any unforeseen issues that may occur.

Term / Week	Objective / Deadline
T1 W1-2	Initial Literature Review & Finalise Plan Project Specification Due (16th October)
T1 W3-6	Phase 1: Implement Transaction Simulator Design user personas, generate transaction data, and test output.
T1 W7-10	Phase 1: Implement & Train Baseline ML Defender Benchmark model performance on clean, simulated data.
Winter Break	Buffer time for delayed objectives Start Final Report
T2 W1-2	Finalise Progress Report Progress Report Due (22nd January)
T2 W3-5	Phase 2: Implement Adversarial Attacker Design and code evasive attack strategies.
T2 W6-9	Phase 2: Run Robustness Evaluation Experiments Collate results and begin analysis. Final Report Plan Due (13th March)
Break	Phase 3: Intensive Write-up Period Finalise Final Report and Project Management Report. Final Report Due (9th April) Project Management Report Due (9th April)
T3 W1-2	Phase 4: Viva Preparation Prepare presentation slides, potential demo, and practice.

Table 1: Project Timetable

5 Resources & Risks

5.1 Resources

The project will be developed using standard software and hardware, with no specialised equipment required.

- **Hardware:** A personal computer with sufficient processing power and memory for training standard machine learning models.
- **Software & Libraries:** The development will be done primarily in Python. Key libraries will include:
 - `Pandas` and `NumPy` for data generation and manipulation.
 - `Matplotlib` and `Seaborn` for creating charts and visualisations of model performance and attack effectiveness.
 - Standard machine learning libraries for implementing the machine learning models. Some examples include `Scikit-learn` and `PyTorch`
- **Data Source:** All transaction data will be synthetically generated by the custom-built simulator developed in Phase 1 of the project. This avoids the ethical and privacy constraints associated with real financial data.
- **Version Control & Documentation:** Source code will be managed with Git and hosted on a private GitHub repository. The project specification, reports, and final dissertation will be written in LaTeX using Overleaf.
- **Storage:** Data sources, articles for reference, and other relevant documentation will be stored and backed up via Google Drive.

5.2 Risks

A number of potential risks have been identified. The table below outlines these risks and the mitigation strategy for each.

Risk	Mitigation Strategy
Simulation Lacks Realism: The synthetic data generated is too simplistic, making the fraud detection task trivial and undermining the validity of the results.	The simulator’s design will be based on statistical properties found in real-world public datasets. The development will be iterative, allowing for refinement if initial results are not realistic enough.
Technical Hurdles in Development: Unforeseen bugs or complexities in developing the simulator or ML pipeline could cause significant delays.	The project will follow an iterative development model, allowing for early detection of issues. Buffer time is explicitly allocated in the project timetable to absorb minor delays.
Loss or Failure of Equipment: Hardware failure could lead to the loss of critical work and code.	All source code and documentation will be regularly backed up to remote cloud repositories (GitHub and Overleaf).
Insufficient Model Performance: The baseline ML models fail to achieve reasonable fraud detection accuracy, making robustness evaluation less meaningful and undermining the validity of adversarial attack testing.	Perform systematic hyperparameter tuning and feature engineering. If performance remains inadequate, revisit dataset selection and preprocessing steps.

Table 2: Project Risks & Mitigation Strategies

6 Legal, Social, Ethical and Professional Issues

This project involves the evaluation of machine learning-based financial fraud detection systems under adversarial conditions. The implementation of adversarial attacks will be carefully designed to ensure that it poses no threat to real-world systems or society. All experiments will be conducted in a controlled, academic environment using only synthetic data and simulated financial scenarios. The attack code will not be capable of targeting or interacting with any external or real financial systems. The project’s intent is clearly documented as defensive research to improve system robustness, not to create malicious tools.

References

- [1] Adversarial learning in real-world fraud detection. *ACM Digital Library*, 2023.
- [2] Financial fraud detection through the application of machine learning. *Nature*, September 2024.
- [3] Long Dang, Thushari Hapuarachchi, Kaiqi Xiong, and Jing Lin. Improving machine learning robustness via adversarial training. *arXiv preprint arXiv:2309.12593*, 2023.
- [4] Prove. Implementing fraud detection for financial institutions. <https://www.prove.com/blog/financial-fraud-detection-challenges>, October 2025.
- [5] Wipro. Making ml models in banking resilient using adversarial attacks, December 2022.